



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Ingegneria del Software: Tecniche Avanzate

CodeSmile



Pre Modifications System Testing Document
Versione 1.0

Team

Choaib Goumri - NF22500223
Mattia Gallucci - NF22500127

Repository GitHub

Anno Accademico 2025–2026

Indice

1	Introduzione	1
1.1	Obiettivi del Testing	1
1.2	Ambiente di Test	2
1.3	Tipologia e Metodologia di Testing	2
2	Test di Sistema	3
2.1	Individuazione dei Parametri e delle Categorie (Category Partition) . . .	3
2.1.1	Parametri di Input e Struttura (F & ST)	3
2.1.2	Parametro Smell (S)	3
2.1.3	Parametri di Configurazione del Tool (CT)	3
2.1.4	Parametri Infrastruttura Web (W)	4
2.2	Scelte e Combinazioni	4
2.2.1	Parametri di Input	4
2.2.2	Parametro Struttura del Progetto	4
2.2.3	Parametro Smell	5
2.2.4	Parametri di Configurazione del Tool	5
2.2.5	Parametri Backend WebApp	5
2.3	Test Frame	6

1 Introduzione

Il presente documento descrive e formalizza l'attività di System Testing eseguita sulla versione iniziale del sistema CodeSmile. Tale attività si colloca nella fase "Pre-Modifications", ovvero prima dell'implementazione delle Change Requests (modifiche evolutive o correttive) pianificate.

1.1 Obiettivi del Testing

L'obiettivo primario di questa sessione di test è duplice:

1. **Validazione Funzionale:** Verificare il corretto funzionamento dei moduli principali (analisi del codice sorgente, rilevamento dei code smell, generazione della reportistica) in uno scenario end-to-end.
2. **Regression Baseline:** Stabilire una base di riferimento solida (baseline) per i futuri test di regressione. Questo garantirà che le funzionalità esistenti non subiscano degradazioni (regressioni) una volta introdotte le nuove feature.

Sebbene CodeSmile supporti diverse modalità di interazione (CLI, GUI), la totalità dei test di sistema è stata eseguita tramite Interfaccia a Riga di Comando (CLI).

Questa scelta metodologica è motivata da tre fattori:

1. La business logic di analisi è condivisa tra tutte le interfacce; validare la CLI valida implicitamente il motore core utilizzato dalla Web App.
2. La CLI permette una maggiore rapidità nell'esecuzione di test batch e nell'automazione degli script.
3. L'isolamento da eventuali problematiche di rendering frontend permette di focalizzarsi sulla correttezza dell'output analitico.

1.2 Ambiente di Test

Per garantire la riproducibilità dei risultati, i test sono stati eseguiti nel seguente ambiente controllato:

- **Sistema Operativo:** Windows 11
- **Interprete Python:** Versione 3.11
- **Gestione Dipendenze:** Librerie installate conformemente al file requirements.txt.
- **Repository:** https://github.com/ChoaibGoumri/smell_ai

1.3 Tipologia e Metodologia di Testing

L'attività è stata condotta adottando un approccio **Black-Box Testing**. Il sistema è stato valutato basandosi esclusivamente sugli input forniti e sugli output osservabili, ignorando la struttura interna del codice.

Per la progettazione dei Test Case è stato applicato il **Category Partition Method**. Questo metodo ha permesso di:

1. Decomporre il dominio degli input in categorie indipendenti.
2. Identificare "scelte" (valori o classi di equivalenza) per ogni categoria.
3. Combinare queste scelte in **Test Frame** che coprano scenari nominali, al limite (boundary) ed erranei.

2 Test di Sistema

Questa suite di test ha l'obiettivo di validare il comportamento complessivo di CodeSmile prima dell'implementazione delle nuove funzionalità.

- **Nota sui test esistenti:** Prima di questa fase, i singoli moduli (motore di analisi, rule checker, estrattori) sono stati verificati tramite unit e integration test.
- **Interfaccia di esecuzione:** Sebbene la business logic sia condivisa, i test vengono condotti principalmente via CLI per facilitare l'automazione, integrando scenari Web/GUI per coprire le criticità evidenziate nelle Change Request.

2.1 Individuazione dei Parametri e delle Categorie (Category Partition)

Utilizziamo il metodo delle partizioni di categoria per isolare i fattori che influenzano l'analisi. I parametri identificati sono:

2.1.1 Parametri di Input e Struttura (F & ST)

- Numero di Progetti (NP): $NP = \{1, > 1\}$.
- Numero di File (NF): $NF = \{0, 1, > 1\}$.
- Estensione File (EF): Specifica se i file sono Python (.py) o altri formati non supportati.
- Struttura Directory (SD): Organizzazione dei file (lineare o annidata/multi-livello).

2.1.2 Parametro Smell (S)

- Numero di Code Smells (NCS): $NCS = \{0, 1, > 1\}$.
- Tipo di Code Smell (TCS): Generico, API-Specific (Pandas, PyTorch, ecc.), Altro.

2.1.3 Parametri di Configurazione del Tool (CT)

- Modalità di Esecuzione (ME): CLI, WebApp/GUI.
- Esecuzione Parallela (EP): Flag booleano (True/False).
- Numero di Walkers (NW): $NW = \{< 5, 5, > 5\}$.

- Resume (RES): Capacità di riprendere un'analisi interrotta.
- Percorso di Output (OUT): Accessibilità della directory di destinazione.

2.1.4 Parametri Infrastruttura Web (W)

- Disponibilità Backend (DB): Stato dei microservizi (Online/Offline).
- Esito Gateway (EG): Risultato delle chiamate API (HTTP 2xx, 4xx, 5xx).

2.2 Scelte e Combinazioni

2.2.1 Parametri di Input

Categoria	Scelte
Numero di File Input (NF)	0 [ERRORE] 1 > 1
Estensione File (EF)	Almeno un file Python (.py) Nessun file .py [ERRORE]

Tabella 2.1: Parametri di input

2.2.2 Parametro Struttura del Progetto

Categoria	Scelte
Numero dei Progetti (NP)	1 > 1
Struttura del Progetto (SP)	Struttura singola Struttura annidata Struttura inaccessibile [ERRORE]

Tabella 2.2: Struttura del progetto

2.2.3 Parametro Smell

Categoria	Scelte
Numero di Code Smells (NCS)	0 1 > 1
Tipo di Code Smell (TCS)	Generico API-Specific Altro/Misto

Tabella 2.3: Parametri smell

2.2.4 Parametri di Configurazione del Tool

Categoria	Scelte
Modalità di Esecuzione (ME)	WebApp/GUI CLI
Esecuzione Parallela (EP)	True False
Numero di Walkers (NW)	<=0 [ERRORE] <5 5 >5
Resume (RES)	True False
Percorso di Output (OUT)	Accessibile Non accessibile [ERRORE]

Tabella 2.4: Configurazione del tool

2.2.5 Parametri Backend WebApp

Categoria	Scelte
Disponibilità Backend (DB)	Tutti online Almeno uno offline [ERRORE]
Esito Gateway (EG)	HTTP 2xx HTTP 4xx HTTP 5xx

Tabella 2.5: Parametri backend

2.3 Test Frame

I casi di test sono stati derivati selezionando valori rappresentativi per ciascuna categoria, rispettando i vincoli di dipendenza tra i parametri. È stato applicato il principio della singola sorgente di errore per ogni test frame.

Per la baseline pre-modifica, gli oracoli dei test prevedono solo l'identificazione delle categorie esistenti e il conteggio assoluto degli errori, senza la metrica di densità introdotta dalla CR5.

Tabella 2.6: Test cases di sistema

Test Case ID	Configurazione (Test Frame)	Oracolo Atteso
TC_01	NF1, EF2, NP1, SP1, NCS0, TCS0, ME2, EP2, NW0, RES2, OUT1, DB0, EG0	Assenza di file Python, terminazione senza errori critici.
TC_02	NF2, EF1, NP1, SP3, NCS0, TCS0, ME2, EP2, NW0, RES2, OUT1, DB0, EG0	Errore di accesso alla struttura del progetto.
TC_03	NF2, EF1, NP1, SP1, NCS0, TCS0, ME2, EP1, NW1, RES2, OUT1, DB0, EG0	Numero di walkers non valido.
TC_04	NF2, EF1, NP1, SP1, NCS0, TCS0, ME2, EP2, NW0, RES2, OUT2, DB0, EG0	Percorso di output non accessibile.
TC_05	NF2, EF2, NP1, SP1, NCS0, TCS0, ME2, EP2, NW0, RES2, OUT1, DB0, EG0	File non supportati ignorati.
TC_06	NF3, EF1, NP1, SP1, NCS1, TCS0, ME2, EP2, NW0, RES2, OUT1, DB0, EG0	Analisi completata: 0 smell.
TC_07	NF3, EF1, NP2, SP1, NCS1, TCS0, ME2, EP2, NW0, RES2, OUT1, DB0, EG0	Analisi multi-progetto completata.
TC_08	NF3, EF1, NP1, SP2, NCS1, TCS0, ME2, EP2, NW0, RES2, OUT1, DB0, EG0	Struttura annidata senza smell.
TC_09	NF2, EF1, NP1, SP1, NCS2, TCS1, ME2, EP2, NW0, RES1, OUT1, DB0, EG0	Rilevato smell generico.
TC_10	NF2, EF1, NP1, SP1, NCS2, TCS2, ME2, EP2, NW0, RES1, OUT1, DB0, EG0	Rilevato smell API-specific.
TC_11	NF2, EF1, NP1, SP1, NCS3, TCS1, ME2, EP2, NW0, RES1, OUT1, DB0, EG0	Più smell generici rilevati.
TC_12	NF2, EF1, NP1, SP1, NCS3, TCS2, ME2, EP2, NW0, RES1, OUT1, DB0, EG0	Più smell API-specific.

Test Case ID	Configurazione (Test Frame)	Oracolo Atteso
TC_13	NF2, EF1, NP1, SP1, NCS3, TCS3, ME2, EP2, NW0, RES1, OUT1, DB0, EG0	Combinazione mista di smell.
TC_14	NF2, EF1, NP1, SP1, NCS2, TCS1, ME2, EP1, NW2, RES1, OUT1, DB0, EG0	Analisi parallela ($NW < 5$).
TC_15	NF2, EF1, NP1, SP1, NCS2, TCS1, ME2, EP1, NW3, RES1, OUT1, DB0, EG0	Analisi parallela ($NW = 5$).
TC_16	NF2, EF1, NP1, SP1, NCS2, TCS1, ME2, EP1, NW4, RES1, OUT1, DB0, EG0	Analisi parallela ($NW > 5$).
TC_17	NF2, EF1, NP1, SP1, NCS1, TCS0, ME1, EP2, NW0, RES0, OUT1, DB1, EG1	WebApp: servizi online.
TC_18	NF2, EF1, NP1, SP1, NCS0, TCS0, ME1, EP2, NW0, RES0, OUT1, DB2, EG0	WebApp: servizio offline.
TC_19	NF2, EF1, NP1, SP1, NCS0, TCS0, ME1, EP2, NW0, RES0, OUT1, DB1, EG2	WebApp: errore 4xx.
TC_20	NF2, EF1, NP1, SP1, NCS0, TCS0, ME1, EP2, NW0, RES0, OUT1, DB1, EG3	WebApp: errore 5xx.